PlacementRAG: Project Overview & Workflow

> **PlacementRAG** is an AI-powered placement assistant tailored for University of Delhi (Faculty of Technology) students. It features an interactive RAG (Retrieval-Augmented Generation) pipeline that lets students ask questions about companies and explore over 80+ real role profiles and their selection processes using semantic search.

🏗 Tech Stack Overview

- **Frontend:** Next.js 15, TypeScript, Tailwind CSS v4, Framer Motion
- **Backend:** FastAPI, Python 3.12, Uvicorn
- **AI & ML:** Google Gemini 2.5 Flash (via `google-genai`), Sentence Transformers (`all-MiniLM-L6-v2`)
- **Database:** FAISS (Local persistent index), SQLite/SQLAlchemy (Relational Data)
- **Authentication:** NextAuth.js (Google OAuth)
- **Event-Driven Services:** Google Cloud Pub/Sub (Gmail Push Notifications)

⚙️ Detailed Workflow & Architecture

The application operates across four primary workflows: offline data ingestion, online role search, online AI chat, and the automated job application tracking system.

1. Data Ingestion & Indexing Pipeline (Offline)

This process builds the foundation of the RAG system by converting raw JSON files into a searchable vector database.

1. **Raw Data:** Over 80+ company role profiles (stored in `Companies_data/` as JSON files).
2. **Preprocessing:** The `index_data.py` script reads the JSON data and applies a custom preprocessing function to format text specifically for embeddings.
3. **Embedding Generation:** It uses the `sentence-transformers/all-MiniLM-L6-v2` model to convert the text into dense mathematical vectors (embeddings).
4. **FAISS Indexing:** The embeddings are loaded into a FAISS local index (`faiss_index.bin`), while the original documents' metadata is stored in a pickle file (`metadata.pkl`).
5. **Result:** Sub-millisecond vector similarity search capabilities across all company documents.

2. AI Chat Assistant Workflow (Online)

This pipeline uses Gemini alongside the FAISS index to answer natural language queries accurately.

1. **User Prompt:** The user types a natural language question (e.g., "What companies hire

for SDE roles above 20 LPA?") in the Next.js `ChatMode` interface.
2. **API Request:** The query is sent via a `POST` request to the backend `/ask/` endpoint.
3. **RAG Service (`rag_service.py`):**
    - The user query is vectorized using the same Sentence Transformer model.
    - The query vector retrieves the top `K` most relevant company contexts from the FAISS database.
    - A structured prompt containing both the user's query and the retrieved context is formed.
4. **LLM Generation:** The constructed prompt is sent to **Gemini 2.5 Flash**.
5. **Response Delivery:** The response is returned to the frontend and elegantly presented to the user.

3. Semantic Role Search Workflow (Online)

This feature allows users to semantically search for specific job roles based on custom filters and natural language tags.

1. **User Search:** The user types queries like "React intern" or "10+ LPA" into the Next.js `SearchMode` interface.
2. **API Request:** A `GET` request is sent to the backend `/search/?query=...` endpoint.
3. **Vector Retrieval (`retriever.py`):**
    - The backend converts the search string into an embedding vector.
    - FAISS calculates the L2 distance between the query vector and the document vectors.
    - The most semantically similar documents are retrieved.
4. **Data Formatting:** The matched records (including their match percentage/score based on L2 distance) are mapped to a UI-friendly dictionary schema.
5. **UI Rendering:** The Next.js frontend renders glassmorphism cards for each result. Clicking a card fetches its full profile via `/search/{role_id}`.

4. Job Application Tracking Workflow (Event-Driven)

This feature allows students to apply for roles directly and leverages the Gmail API to automatically track recruiter replies.

1. **Email Dispatch:** Students draft applications in the `ApplicationsMode` interface. The backend uses their Google OAuth token to send the email and subscribes their inbox to a Google Cloud Pub/Sub topic via the Gmail `watch()` API.
2. **Database Tracking:** The thread ID and application details are saved in the relational database (SQLite via SQLAlchemy).
3. **Webhook Notifications:** When a recruiter replies, Gmail pushes a notification to the Pub/Sub topic, which triggers the backend `/webhooks/gmail` endpoint.
4. **History Sync & Filtering:** The backend fetches the latest inbox changes, matching them to tracked application threads while filtering out emails sent by the student.
5. **AI Status Classification:** The new recruiter reply is passed to Gemini 2.5 Flash, which categorizes the email as `Selected`, `Rejected`, `Interview`, `Online Assessment`, or `Unclear`. The database and frontend UI are updated instantly.

🧩 Component Breakdown

Backend (`/backend`)

- **`main.py`:** Initializes the FastAPI app, configures CORS, initializes the database, and mounts all routers.
- **`app/routes/`:**
  - `ask.py` & `search.py`: Handle RAG and vector queries.
  - `applications.py`: Handles sending application emails and fetching tracking history.
  - `webhooks.py`: Receives Google Cloud Pub/Sub events for inbox changes.
- **`app/services/`:**
  - Contains core logic (`rag_service.py`, `retriever.py`).
  - `email_service.py`: Gmail API integration (send email, watch inbox, fetch history).
  - `webhook_handler.py`: Parses push notifications and matches threads.
  - `classifier.py`: Prompts Gemini to categorize recruiter replies.
- **`app/db/`:** SQLAlchemy setup (`database.py`) and schema definitions (`models.py`).
- **`index_data.py`:** The primary ETL pipeline script for managing updates to company vector data.

Frontend (`/frontend`)

- **`app/page.tsx`:** The root layout utilizing a sidebar navigation to switch between the "Chat" and "Search" workspaces. Features mobile responsiveness and frosted glass styling.
- **`components/`:**
  - **`ChatMode`**: Implements the user-facing interface for interacting with the AI Assistant.
  - **`SearchMode`**: Implements the user-facing interface for searching and browsing role cards.
  - **`ApplicationsMode`**: The tracking dashboard where students can draft emails and monitor AI-classified application statuses.
  - **`AuthLogin`**: Secures access and requests Gmail modification permissions via Google NextAuth.

> [!TIP] **Updating the Database:** Whenever new JSON role files are added to `Companies_data/`, simply re-run `python3 index_data.py` in the backend environment to update the `faiss_index.bin` and keep the semantic search up-to-date.